PERFORMANCE TEST REPORT

ShopLite e-commerce API: load test, bottleneck analysis and tuning verification

Prepared by **Michael Youssef**, Performance & Load Testing (JMeter) | ISTQB CTFL

Portfolio sample. The system under test is a demo e-commerce API built for this exercise and tested in a lab environment. Every figure in this report comes from real test runs; no client data is used.

1. Executive summary

Verdict	**Baseline: FAIL** at the 300-user target. The API saturates at about **140 req/s** from 200 users; at 300 users p95 response time reaches **1000 ms** (target 500 ms). **After tuning: PASS.** Throughput scales linearly to **236 req/s** at 300 users with p95 of **37 ms**.

- **Capacity before tuning:** about 100 concurrent users within the response-time target. Beyond that, extra users only add waiting time, not throughput.
- **Root cause:** the database connection pool (3 connections). Every transaction slowed down by the same amount, which points to a shared resource rather than one slow endpoint. Checkout holds a connection longest (60–120 ms), so it drives pool occupancy.
- **Fixes verified:** pool raised to 24 connections, product-detail caching, faster checkout query, indexed search. Result at 300 users: **+67% throughput** and **p95 down 96%**.
- **Errors:** 0 failed requests in both runs (23,105 and 29,299 requests).

Results at each load step (steady state)

Virtual users	Baseline req/s	Baseline median	Baseline p95	Tuned req/s	Tuned median	Tuned p95	Target met?
50	39.7	21 ms	83 ms	39.4	18 ms	42 ms	Both
100	78.5	22 ms	82 ms	79.3	17 ms	40 ms	Both
200	138.1	195 ms	355 ms	158.2	17 ms	39 ms	Both
300	141.2	870 ms	1000 ms	235.8	17 ms	37 ms	Tuned only

Target: p95 under 500 ms and error rate under 1% at 300 concurrent users. Steady state = last two-thirds of each 60-second step.

2. Objectives and scope

- Find the maximum load the API can sustain while meeting the response-time target.
- Locate the bottleneck and confirm the fix with a repeat test under identical load.
- In scope: the six user transactions below. Out of scope: front-end rendering, third-party payment gateway, soak and spike tests (recommended as next steps).

3. Test approach

Workload model

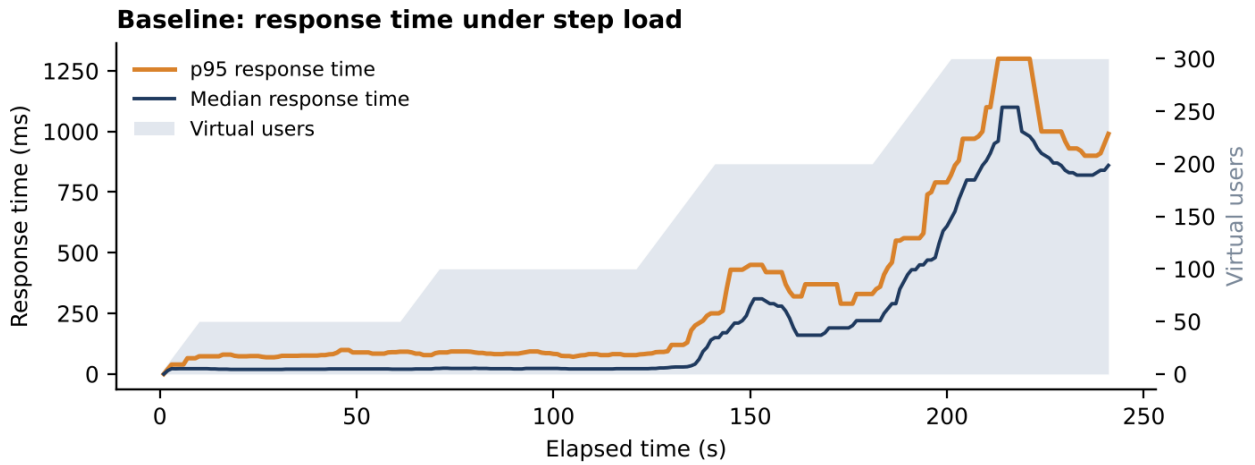
Transaction	Endpoint	Weight	Notes
01 Login	POST /api/login	once per user	On session start
02 Browse catalogue	GET /api/products?page=n	5	Random page 1–50
03 Product details	GET /api/products/{id}	4	Random product 1–2,000
04 Search	GET /api/search?q=	2	Random term
05 Add to cart	POST /api/cart	2	JSON body
06 Checkout	POST /api/checkout	1	Write-heavy

Think time between actions: 0.5–2 s (random), to reflect real shopper pacing.

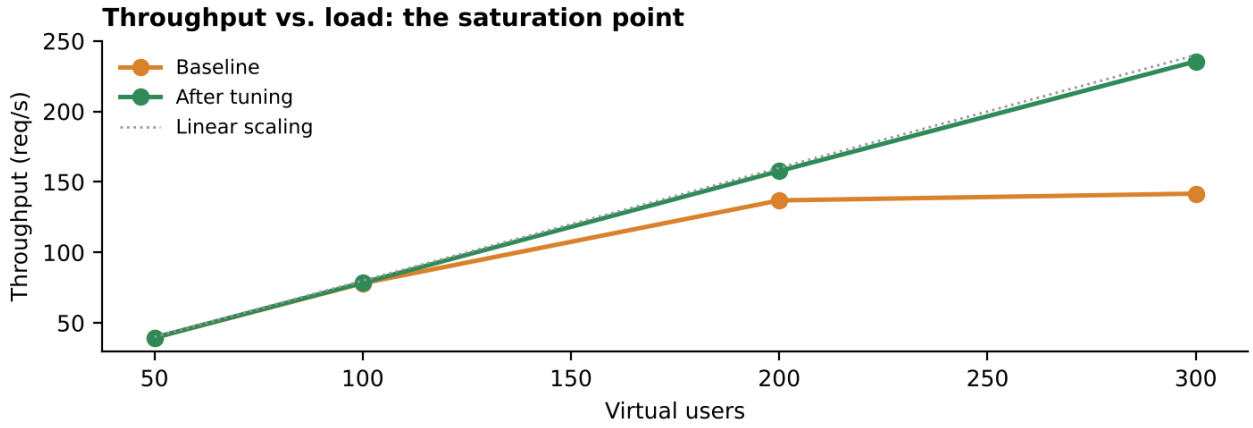
Load profile and environment

Item	Value
Profile	Step load: 50 → 100 → 200 → 300 virtual users, 60 s per step, ramp 5 users/s
Duration	4 minutes per run; baseline and tuned runs under identical load
System under test	ShopLite demo API (Python/FastAPI), single instance, lab environment
Load tool	Locust 2.47 (Python). The same scenario is provided as a JMeter test plan (.jmx) in the repository.
Metrics	Throughput, median and p95 response time, error rate, per transaction and over time

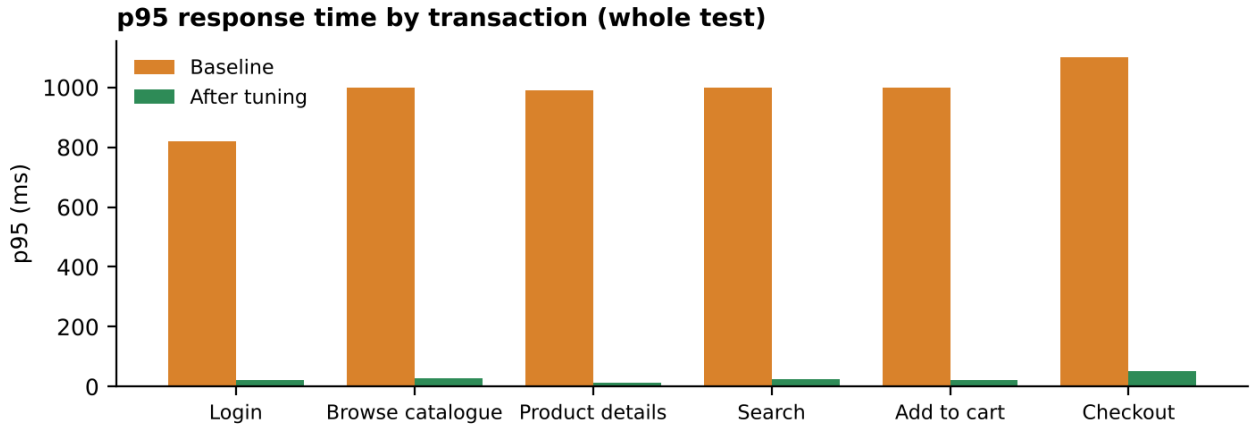
4. Results



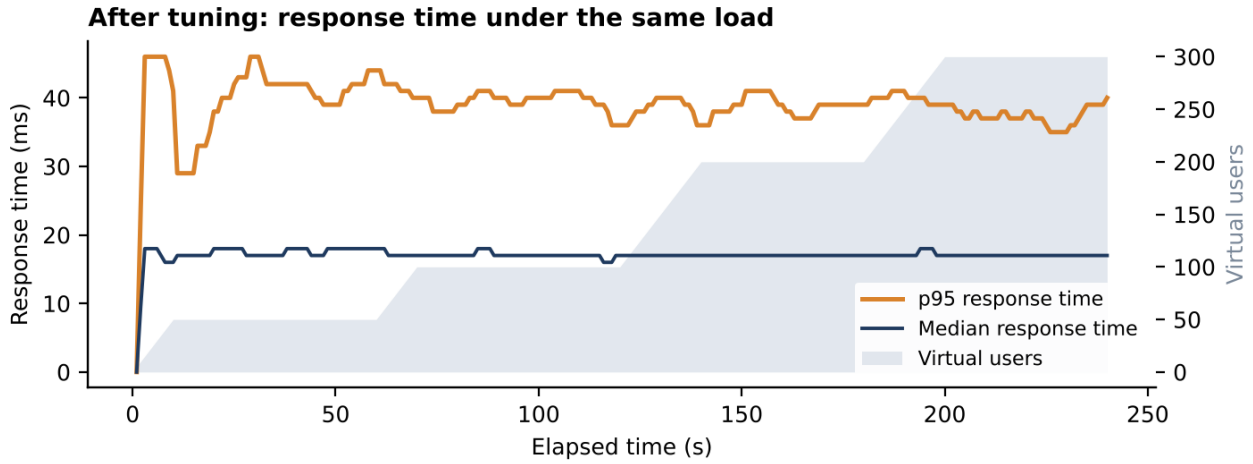
Baseline: response time is flat up to 100 users, starts climbing during the 200-user step, and passes 1 second at 300 users.



Throughput stops growing after 200 users in the baseline: the classic saturation point. After tuning it follows the linear-scaling line.



All six transactions degrade by a similar amount in the baseline, including lightweight ones like product details. That pattern points to a shared resource.



After tuning: median and p95 stay flat across all four load steps.

Per-transaction results (whole test)

Transaction	Requests (B / T)	Avg ms (B / T)	p95 ms (B / T)	p99 ms (B / T)	Errors
01 Login	300 / 300	232 / 18	820 / 23	960 / 27	0 / 0
02 Browse catalogue	8,105 / 10,214	362 / 20	1000 / 27	1200 / 28	0 / 0
03 Product details	6,503 / 8,361	349 / 4	990 / 13	1200 / 15	0 / 0
04 Search	3,300 / 4,227	366 / 19	1000 / 24	1200 / 25	0 / 0
05 Add to cart	3,285 / 4,118	360 / 18	1000 / 23	1200 / 24	0 / 0
06 Checkout	1,612 / 2,079	422 / 43	1100 / 52	1200 / 53	0 / 0
All transactions	**23,105 / 29,299**	**361 / 17**	**1000 / 39**	**1200 / 50**	**0 / 0**

B = baseline, T = after tuning. The tuned run completed more requests in the same time because it was not queueing.

5. Bottleneck analysis

- **Symptom 1, flat throughput:** going from 200 to 300 users added 50% more load but only about 2% more throughput. Requests were waiting, not being processed.
- **Symptom 2, uniform slowdown:** product details (5–12 ms of database work) slowed down as much as checkout. When cheap and expensive calls degrade together, they are queuing for the same resource.
- **Diagnosis:** a database connection pool of 3. At about 140 req/s with an average of 20 ms of database time per request, the pool is fully busy, so every new request waits for a free connection (Little's law: 140 × 0.02 s ≈ 3 connections in use).
- **Contributing factors:** checkout holds a connection for 60–120 ms; product details are read from the database on every call; search does an unindexed scan that also uses CPU.

6. Changes applied and verified

Change	Why	Effect seen in the repeat test
Connection pool 3 → 24	Remove the queue for connections	Throughput scales linearly to 300 users
Cache product details	Avoid a database call for hot items	Product details avg 349 → 4 ms
Optimise checkout query	Shorter connection hold time	Checkout p95 1100 → 52 ms
Index search	Remove full scan	Search p95 1000 → 24 ms

7. Recommendations and next steps

- Monitor connection-pool usage and wait time in production, with an alert at 80% usage.
- Run a 2-hour soak test at 200 users to check for memory leaks and slow degradation.
- Run a spike test (0 → 300 users in 10 s) to check recovery behaviour.
- Repeat the test on a production-sized environment before go-live; lab numbers show the trend, not the exact production capacity.
- Add the test to the CI pipeline as a short smoke-load check, so regressions are caught before release.

Appendix: how to reproduce

The test plan, load scripts, raw CSV results and chart scripts are in the accompanying repository. Run the API, then execute the Locust file or the JMeter .jmx plan against it with the same step profile.